ELI — The Complete User Manual

Plain-English Edition · with flowcharts

Contents

1. What ELI is (in one minute) 2
2. How ELI works when you ask it something 2
3. Getting started 4
4. Your first hour with ELI 6
5. Talking to ELI 6
6. A tour of the window 7
7. What ELI can do — by everyday task 7
8. ELI remembers you (memory) 8
9. ELI learns your routines (habits) 9
10. ELI helps before you ask (proactive) 11
11. Voice, wake word, and dictation 11
12. Seeing your screen and images (vision) 11
13. Using ELI from your phone 12
14. Models: picking, switching, and training your own 14
15. How hard ELI thinks (reasoning modes) 15
16. Settings you'll actually touch 15
17. Privacy & staying offline 16
18. Troubleshooting 16
19. Quick phrasebook (cheat sheet) 18
20. Glossary 18
21. Appendix A — Under the hood: how ELI thinks (the full pipeline) 18
22. Appendix B — Under the hood: how ELI remembers (the full memory system) 22

This manual assumes **zero** technical background. You talk to ELI in normal English — by **typing** in the chat box or **speaking** — and it does things on your computer, answers questions, remembers what matters to you, and learns your routines. Everything runs **on your own machine**; nothing is sent to the cloud.

> **The one rule worth knowing:** ELI is **offline by default**. It only touches the internet when *you* ask for something that needs it (a web search, the news, downloading a model). Your conversations, files, and memories never leave your computer.

How to read this manual: - **Just want to use it?** Read §1–§5 and the cheat sheet (§18). That's enough to be productive. - **Want the whole picture?** Read straight through — every feature is here, in plain terms. - **[Anyone] / [A bit technical] / [Advanced] tags** at the start of some sections mean *anyone / a little technical / advanced*.

---

1. What ELI is (in one minute)

ELI is a **personal AI assistant that lives on your computer**. Think of it as a capable helper that can:

- **Talk with you** like a knowledgeable person — by text or voice.
- **Operate your computer** — open apps, play music, manage windows, type for you.
- **Look at things** — your screen, images, PDFs, spreadsheets — and explain them.
- **Write for you** — documents, notes, code, scripts.
- **Remember** what matters about you, and **learn** your daily routines.
- **Act on its own** when helpful — a morning briefing, a heads-up, a suggestion.

The big difference from cloud assistants: **it's yours**. It runs locally, stays private, has no subscription, and you can even retrain its "voice" yourself (§14).

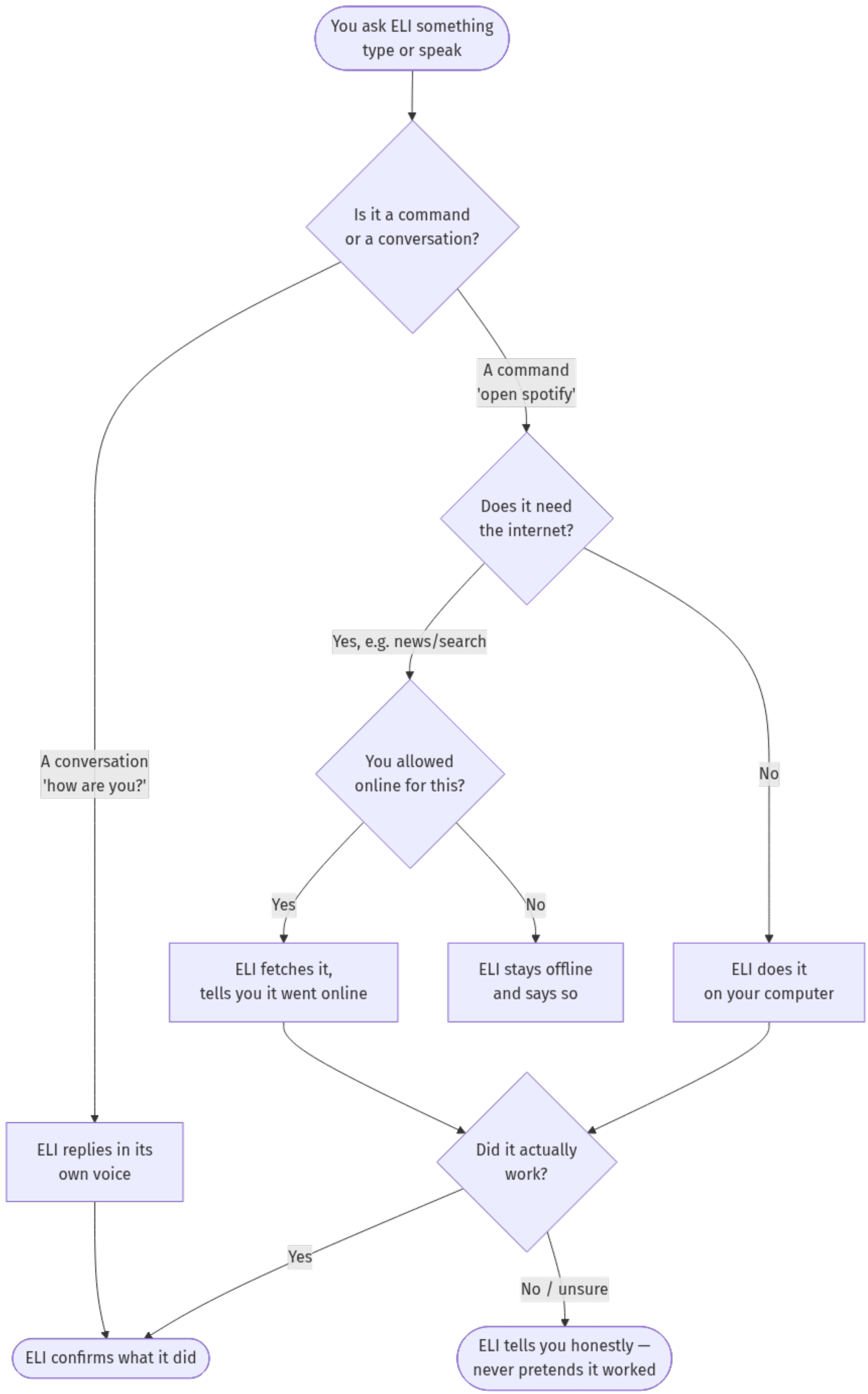
Here's the whole thing at a glance:

---

2. How ELI works when you ask it something

[Anyone] *Anyone — this is the mental model that makes everything else click.*

When you say or type something, ELI follows the same honest loop every time:


You ask ELI something
type or speak
Is it a command
or a conversation?
A command
'open spotify'
Does it need
the internet?
Yes, e.g. news/search
You allowed
online for this?
Yes
No
No
A conversation
'how are you?'
ELI fetches it,
tells you it went online
ELI stays offline
and says so
ELI does it
on your computer
ELI replies in its
own voice
Did it actually
work?
Yes
No / unsure
ELI confirms what it did
ELI tells you honestly —
never pretends it worked

The promise in that diagram: ELI **never fakes an action**. If it says it played a song or fixed a file, it really did — and if it couldn't, it says so plainly instead of bluffing. This is built into the software, not just good manners.

---

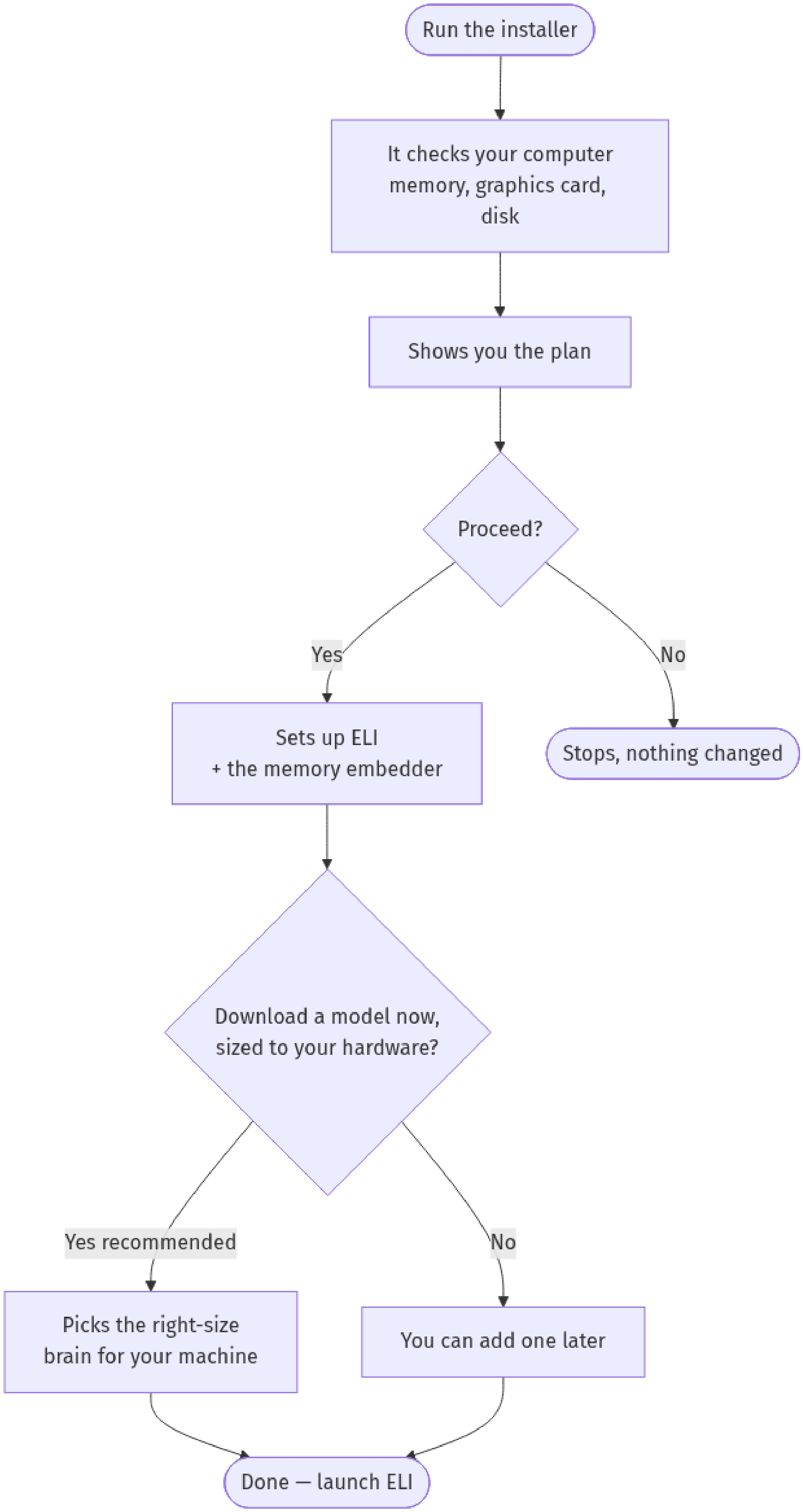
3. Getting started

[A bit technical] *A little technical, but only once.*

Installing

You run a single installer and answer a couple of friendly questions.

- **Windows:** run `install.ps1`
- **Mac & Linux:** run `install.sh`


Run the installer
It checks your computer memory, graphics card, disk
Shows you the plan
Proceed?
Yes
No
Sets up ELI + the memory embedder
Stops, nothing changed
Download a model now, sized to your hardware?
Yes recommended
No
Picks the right-size brain for your machine
You can add one later
Done — launch ELI

The installer also fetches a tiny **embedder** model automatically (it powers ELI's memory), so you don't have to think about it. Everything ELI needs ends up in place — no extra steps.

First launch — the quick interview

The very first time you open ELI, it doesn't know you yet, so it asks **three short questions** (your name, what you do, how you like to be spoken to). Ten seconds, and you can **skip it** entirely. From there, ELI builds its understanding of you naturally as you talk.

Opening ELI

- **Desktop app (full experience):** launch **ELI Pro** from your apps menu, or run `scripts/eli_launch.sh` (Mac/Linux) / the desktop shortcut (Windows).
- **From your phone:** see §13.

---

4. Your first hour with ELI

[Anyone] A gentle on-ramp. Try these in order — each builds confidence.

1. **Say hello.** Type `how are you?` — get a feel for ELI's voice.
2. **Ask what it can do.** `what can you do?` lists its capabilities.
3. **Open something.** `open the file manager` or `open firefox`.
4. **Play music.** `play lo-fi on YouTube`, then `pause`, then `skip`.
5. **Look at your screen.** `what's on my screen?`
6. **Tell it a fact.** `remember that my dog's name is Rufus`. Later: `what's my dog's name?`
7. **Set a reminder.** `set a timer for 5 minutes`.
8. **Get a briefing.** `what's the news?` (this one goes online — ELI will say so).
9. **Check on ELI itself.** `what are you running on?` shows its model and hardware.
10. **Go hands-free.** Say the wake word **"computer"**, then a request.

By the end you've touched conversation, control, vision, memory, reminders, the web, voice, and introspection — the whole surface.

---

5. Talking to ELI

There are two ways, freely mixed:

- **Type** in the chat box and press Enter.
- **Speak** — click the microphone, or say the **wake word** (default **"computer"**) then your request. Change the wake word to anything (§11).

You don't need special phrasing. Talk normally:

> "open spotify and play some lo-fi" "what's on my screen?" "remember that my sister's name is Anna" "set an alarm for 7am"

Chaining: ask for several things at once — > "close steam and set an alarm for 7am" > "open spotify then play Juicy by Notorious B.I.G."

If ELI isn't sure what you meant, it asks — it won't guess wildly or pretend. And if a request needs the internet, it tells you before going online.

---

6. A tour of the window

[Anyone] ELI Pro is organised into **tabs**. You'll mostly live in **Chat**; here's the rest in plain terms:

Tab	What it's for
Chat	Talk to ELI, by voice or text. Home base.
Memory	Browse what ELI remembers about you; add or remove facts.
Habits	Routines ELI has learned; turn on/off, edit times, add your own.
Tasks	Scheduled & overnight jobs and their status.
Proactive	Controls for ELI's "act on its own" behaviour.
Report Builder	Generate longer documents/reports with sources.
Labs	Power-user tools — image generation, code tools, projects, evaluations.
Settings	Model, hardware, voice, and privacy options (§16).

You rarely need the tabs directly — almost everything is reachable by **asking** in Chat: "show my habits", "what do you remember about me", "show background jobs", and ELI opens or reports the right thing.

---

7. What ELI can do — by everyday task

[Anyone] Real things you can say. ELI understands many phrasings — these are examples.

Music & media

- "play lo-fi on YouTube" · "play Juicy by Notorious B.I.G. on Spotify"
- "pause" · "skip" · "next song" · "previous track" · "shuffle" · "repeat this song"
- "what's playing?" · "stop the music" · "skip the ad" (YouTube, when skippable)

Apps & windows

- "open spotify" · "launch firefox" · "open the file manager" · "open github.com"
- "close chrome" · "focus the browser" · "switch to workspace 2"
- "tile windows" · "maximise the window" · "show desktop" · "next window"

Volume, typing, mouse

- "volume up" · "mute" · "set volume to 30"
- "type hello world" · "press enter" · "left click" · "move the mouse up"

Files & notes

- "create a file notes.txt" · "make a folder called projects" · "read notes.txt"
- "list the files in ~/Documents" · "write a note saying buy milk" · "search notes for budget"
- "what's in my clipboard?" · "summarise report.docx" · "convert report.md to pdf"

Writing, documents & code

- “write a report on renewable energy” · “generate a document about X”
- “write a bash script to monitor the GPU” · “solve this: implement a function that …”
- “fix the bugs in foo.py” · “examine eli/memory/memory.py for errors” → ELI scans, **offers** fixes, and applies them only if you say “yes, fix it”. · “show the diff”

Screen, images, PDFs, spreadsheets

- “what’s on my screen?” · “find the submit button on screen” · “take a screenshot”
- “describe cat.jpg” · “read the text in image.jpg” (OCR)
- “summarise report.pdf” · “analyse data.csv” · “watch my screen” / “stop watching”

Information

- “what time is it?” · “what’s the date?” · “weather in Wexford”
- “what’s the news?” / “catch me up” → a **synthesised** briefing, not a raw dump
- “search the web for X” (goes online — ELI tells you) · “gpu status” · “system stats”

Reminders, timers, calendar, scheduling

- “set an alarm for 7am” · “set a timer for 10 minutes”
- “add an event tomorrow at 3pm” · “list my events” · “start a pomodoro”
- “research the best solar inverters overnight” · “build me a script at 2am” → **background jobs**
- “show background jobs” · “check job 5”

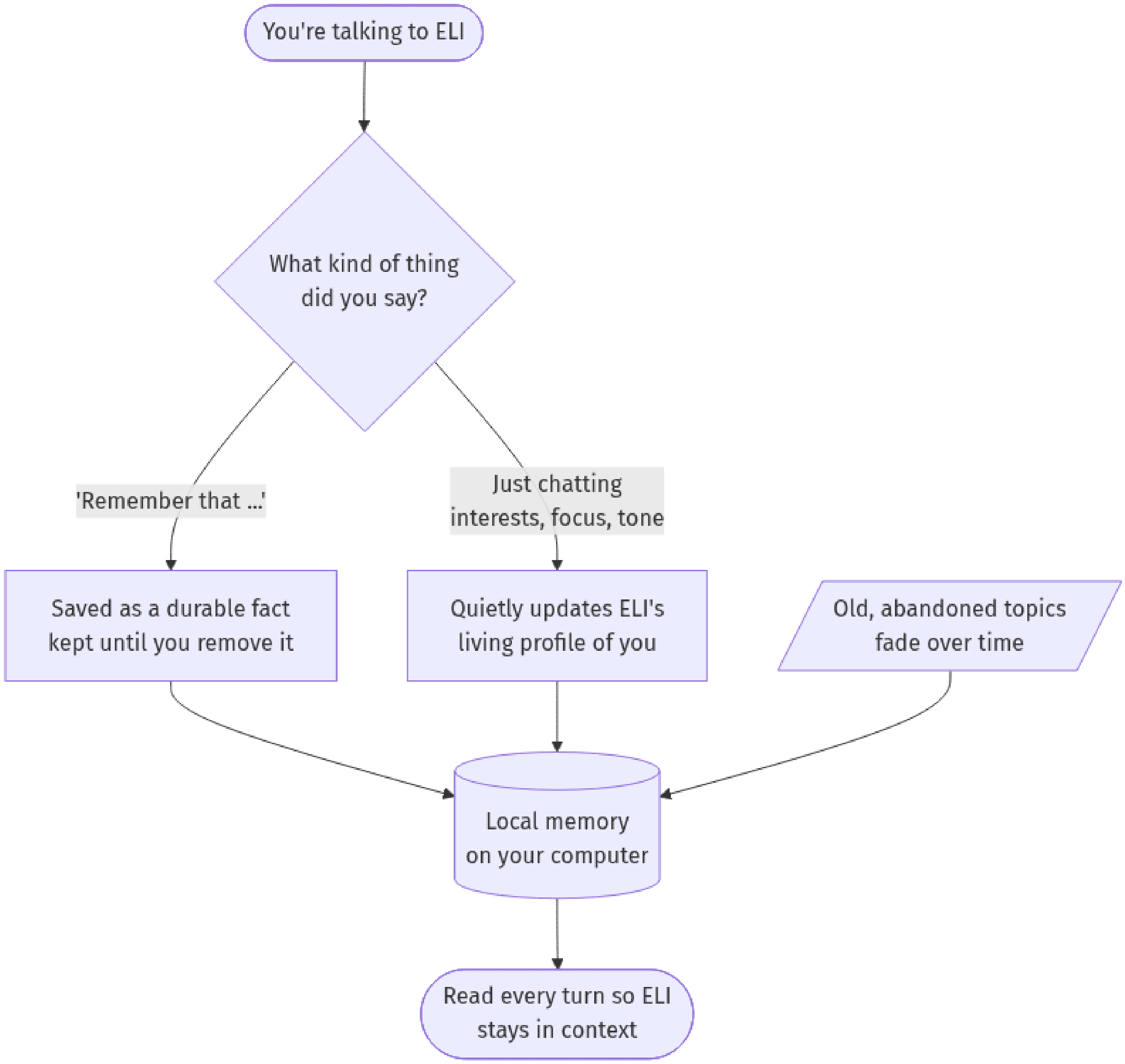
Asking ELI about itself (honest introspection)

- “what are you running on?” (model, context, GPU) · “how does your memory work?”
- “what do you know about me?” · “what have you been working on?” · “status”

---

8. ELI remembers you (memory)

[Anyone] ELI builds a private, evolving understanding of **who you are** and reads it every time you talk, so you never repeat yourself.



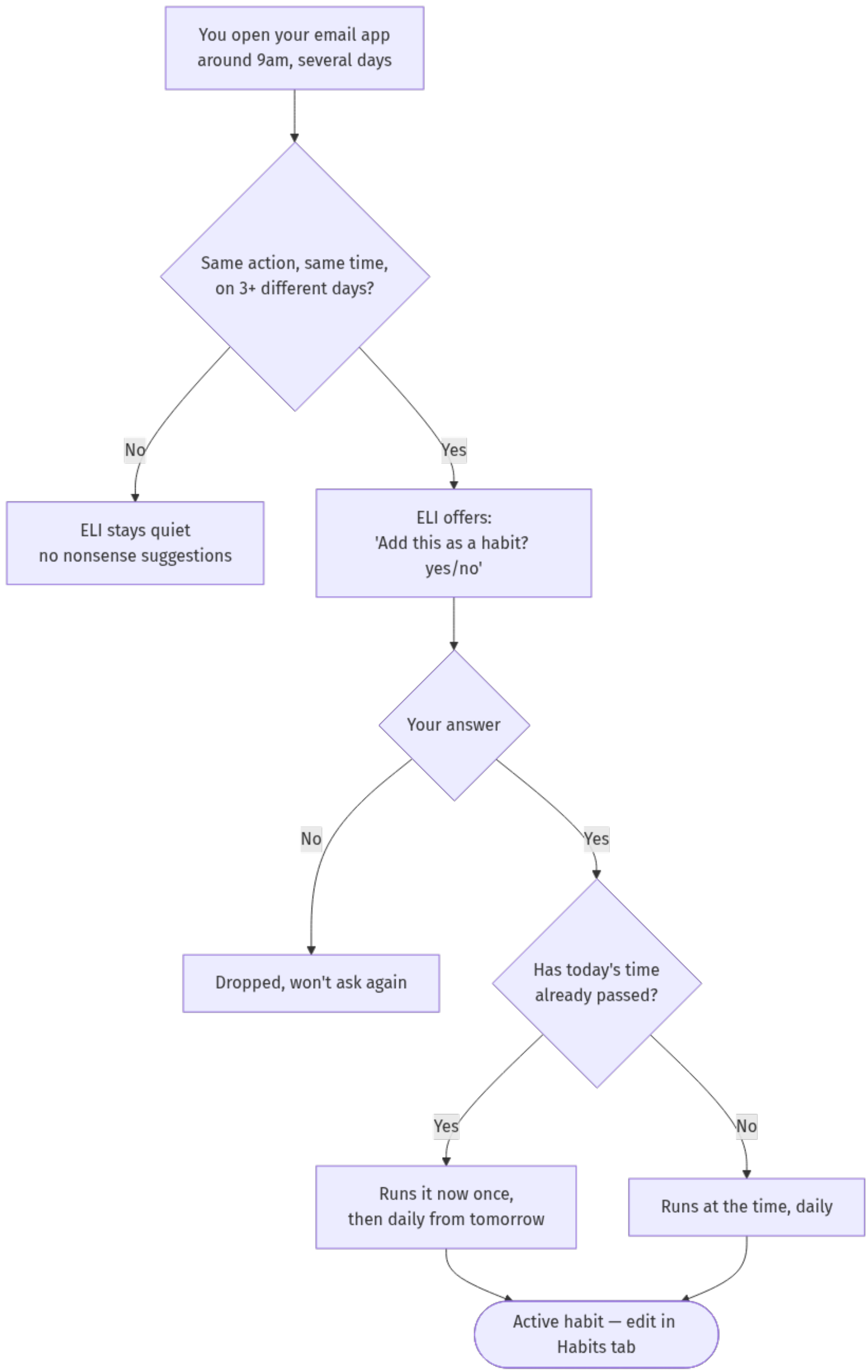
Tell it something to keep: "remember that my sister's name is Anna" · "my name is Alex" **Ask what it knows:** "what do you remember about my car?" · "what do you know about me?" · "explain everything you know about me and where it's stored" (a sourced, detailed answer).

Everything is stored **locally** in small private databases. Review and prune it in the **Memory** tab. A brand-new install is a **blank slate** — ELI knows nothing about you until you talk.

---

9. ELI learns your routines (habits)

[Anyone] If you do the same thing at roughly the same time on **several different days**, ELI notices and **offers** to automate it. It never activates anything without your "yes".


You open your email app around 9am, several days
Same action, same time, on 3+ different days?
No
Yes
ELI stays quiet no nonsense suggestions
ELI offers: 'Add this as a habit? yes/no'
Your answer
No
Yes
Dropped, won't ask again
Has today's time already passed?
Yes
No
Runs it now once, then daily from tomorrow
Runs at the time, daily
Active habit — edit in Habits tab

Manage everything in the **Habits** tab (add, edit times, turn off), or say “show my habits”. The key safeguards: it only proposes routines you’ve **genuinely repeated across days**, and it **always asks first**.

---

10. ELI helps before you ask (proactive)

[Anyone] With **proactive mode** on, ELI can do helpful things on its own — a **morning briefing**, a gentle suggestion, or a goal it thinks would help. It’s conservative and easy to control:

- “start proactive mode” / “stop proactive mode” · “proactive status”
- “morning report” / “give me my briefing” (news, weather, your agenda)
- “do you have any proposals for me?” / “what’s on your agenda?”

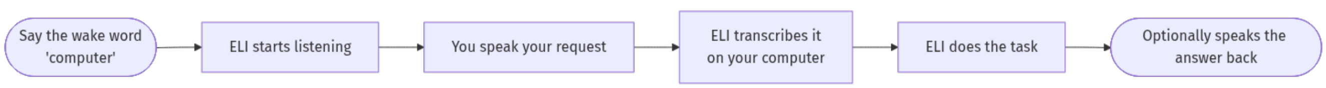
Schedule real unattended work too: > “research X overnight” · “generate tests for your code tonight”

These appear in the **Tasks** tab and survive restarts.

---

11. Voice, wake word, and dictation

[Anyone] ELI is fully usable hands-free.



- **Wake word:** say **“computer”** then your request. Change it: “change the wake word to athena”. ELI trains its listener on the new word automatically — works even over background music.
- **Personalise to your voice:** “enroll my wake word” (records you saying it and adapts).
- **Dictation:** “start dictation” / “stop dictating” — speak and ELI types into the active field.
- **Transcribe a recording:** “transcribe audio.wav”
- **ELI speaking back:** “say good morning”
- **Teach ELI your voice & tone:** “train my voice” — it learns your pitch/energy and even emotion cues, then adapts how it speaks to you.
- **If voice misbehaves:** “run voice diagnostics”.

---

12. Seeing your screen and images (vision)

[Anyone] ELI can actually **look**:

- **Your screen:** “what’s on my screen?” · “what does this error say?” · “find the submit button”
- **Image files:** “describe cat.jpg” · “read the text in this image” (OCR)
- **Documents:** “summarise report.pdf” · “analyse data.csv” · “analyse the pdfs in that folder”
- **Continuous watching:** “watch my screen” (and “stop watching”)

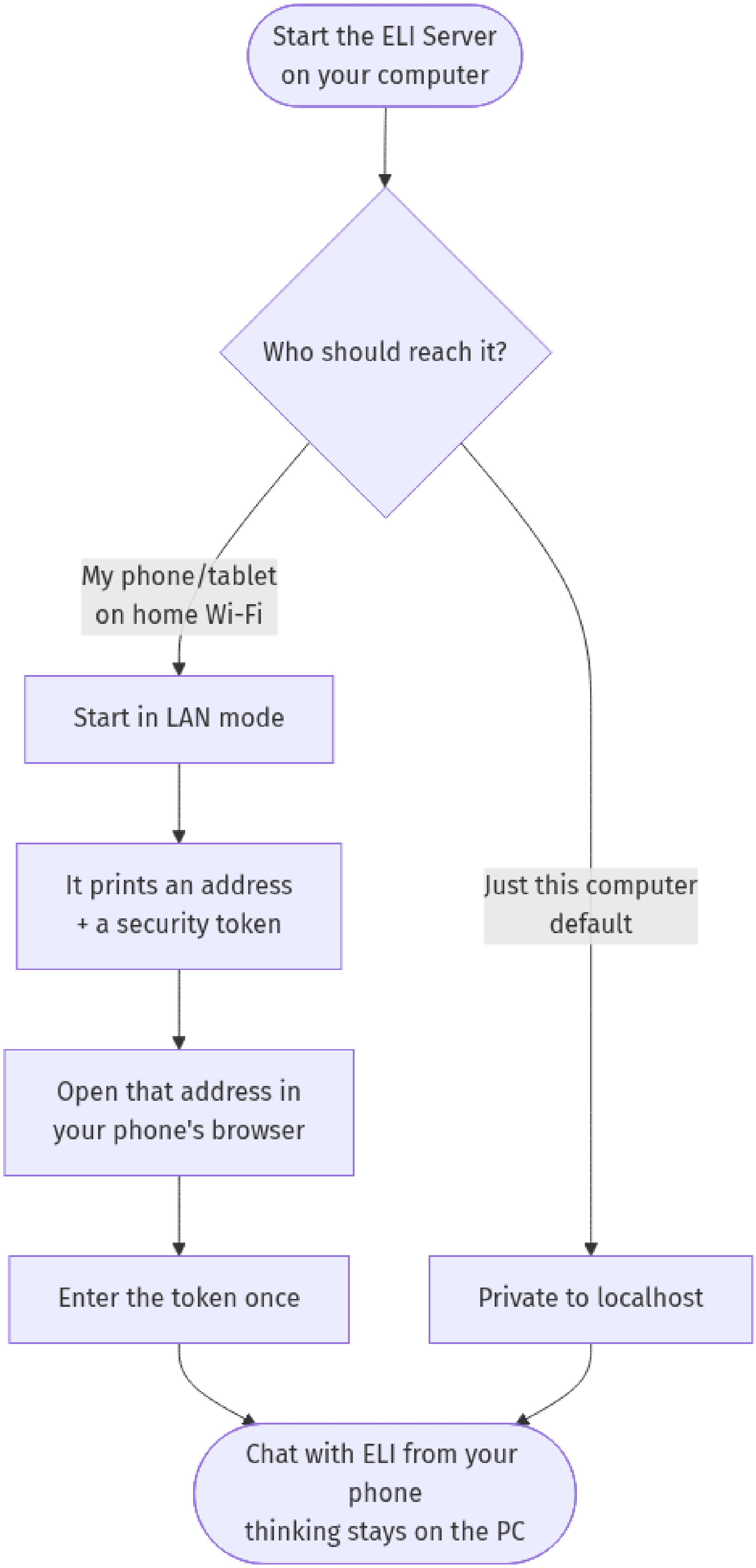
The first time you use vision, ELI may fetch a small vision model (you can pre-download it — §14). With a webcam set up, ELI even supports **eye-tracking control**: “enable gaze control”, “calibrate gaze”, then click where your eyes rest.

---

13. Using ELI from your phone

[A bit technical] *Slightly technical, but only at setup.*

ELI includes a built-in **web app**. Start the server on your computer and chat with ELI from your **phone or tablet’s browser** over your home Wi-Fi — while all the actual thinking stays on your computer.


Start the ELI Server
on your computer
Who should reach it?
My phone/tablet
on home Wi-Fi
Just this computer
default
Start in LAN mode
It prints an address
+ a security token
Open that address in
your phone's browser
Enter the token once
Private to localhost
Chat with ELI from your
phone
thinking stays on the PC

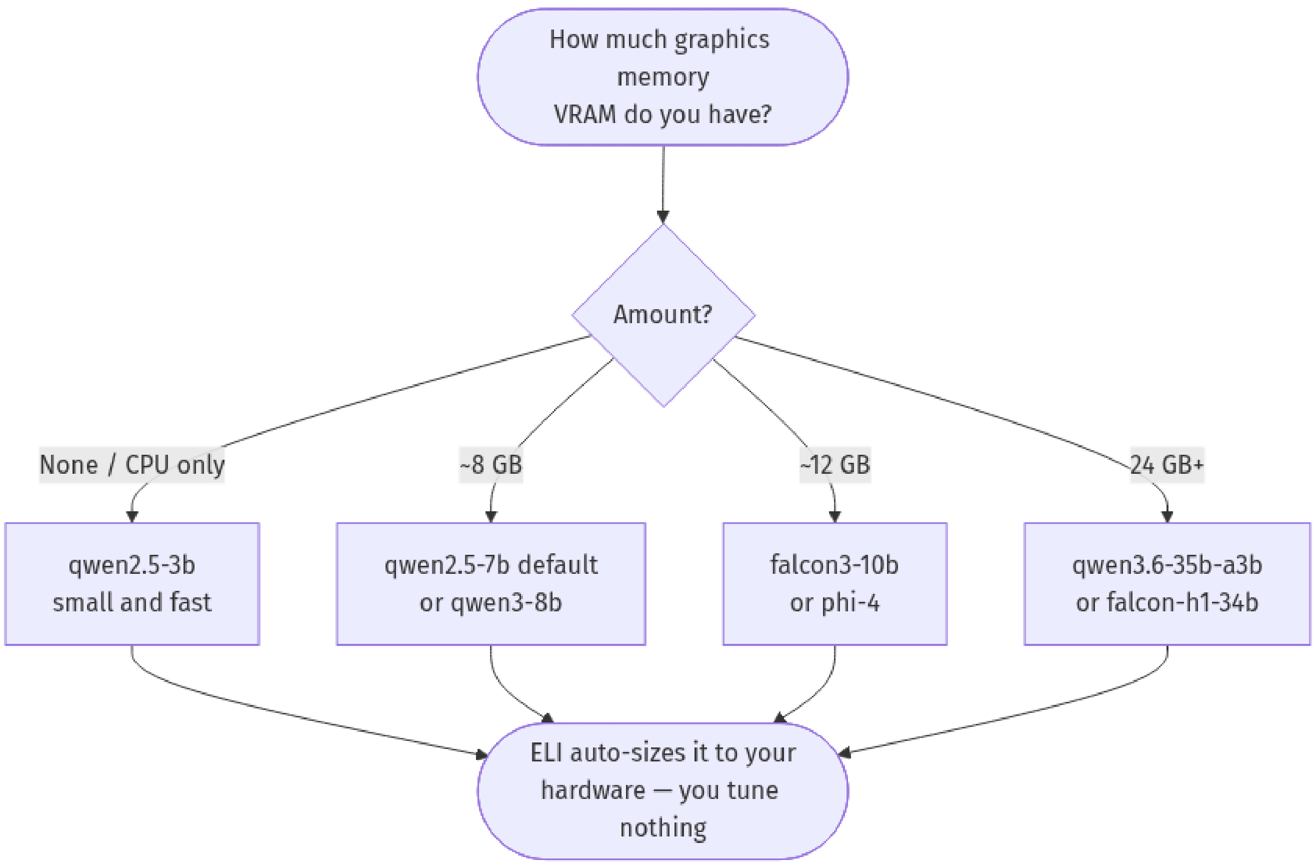
- Start it with the **ELI Server** desktop shortcut, or `scripts/eli_serve.sh` (Mac/Linux) / `scripts/eli_serve.ps1` (Windows). Full details: `docs/SERVER_AND_WEB_APP.md`.
- Over the phone you get chat, memory, search, news, documents, scheduling, status — everything except physically controlling *that PC's* screen/mouse/camera (those run on the host).

---

14. Models: picking, switching, and training your own

[A bit technical] The **model** is the "brain" ELI thinks with — a file on your computer. Bigger = smarter but needs more graphics memory (VRAM). ELI picks a suitable one at install; change it anytime.

Which model should I get?



Downloading

- **Pick from a menu (choose any number of models):** `python -m eli.core.model_download --choose` — this is what the installer uses; you tick the ones you want (or `all` / `auto` / none).
- Let ELI choose one best-fit for your hardware: `python -m eli.core.model_download --auto`
- See the menu without downloading: `python -m eli.core.model_download --list`
- Grab a specific one: `python -m eli.core.model_download qwen2.5-7b`

You're not limited to one — download several and switch between them anytime (§"Switching models").

Command name	Model	Roughly needs
`qwen2.5-3b`	small & fast	4 GB graphics / or CPU
`qwen2.5-7b` *(default)*	great all-rounder	8 GB graphics
`qwen3-8b`	newer, big context, good reasoning	8 GB graphics
`falcon3-10b`	a step up	12 GB graphics
`phi-4`	strong reasoning for its size	12 GB graphics
`qwen3.6-35b-a3b`	very capable (mixture-of-experts)	24 GB / or slow on CPU
`falcon-h1-34b`	largest option	24 GB / or slow on CPU

You can also just **drop any `.gguf` file** into the `models/` folder and ELI finds it.

Switching models

In **Settings → Model** (or "Model menu / Auto Detect"), pick the file. ELI sizes it to your hardware automatically — you don't tune anything.

[Advanced] Training your own ELI-flavoured model (advanced, optional)

You can fine-tune a model so it speaks in **ELI's voice** out of the box.

-> **See `blueprints/finetuning_guide.md`** — a full step-by-step from choosing a base model, through extracting a voice dataset and training, to producing a ready-to-load `.gguf`. Crucially, ELI's **personality stays live** (never frozen into the model); training only shapes the *manner* of speech.

---

15. How hard ELI thinks (reasoning modes)

[Anyone] ELI has five effort levels. Faster modes answer quickly; deeper modes think longer and use more of ELI's internal helpers for hard problems.

Mode	Feel	Use it for
quick	snappy	greetings, simple commands, quick facts
normal	balanced (default)	everyday questions and tasks
advanced	more thorough	tricky, multi-step asks
research	digs deep	gathering and synthesising a lot
expert	maximum effort	the hardest problems

Ask "what reasoning mode are you in?" or "explain all your reasoning modes". For most people the default is right — ELI also deepens on its own when a problem clearly needs it.

---

16. Settings you'll actually touch

[A bit technical] Most people never need these. The few that matter:

- **Model** — which brain ELI uses (§14).
- **Voice** — wake word, text-to-speech on/off, microphone.
- **Hardware / startup** — ELI auto-tunes; advanced users can nudge how much graphics memory to hold back, or how much context to use. Leave alone unless you know why.
- **Privacy** — ELI is offline by default; this is where anything network-related lives.

Two graphics cards? ELI can use both — set in the GPU profile settings; single-card just works.

---

17. Privacy & staying offline

[Anyone]

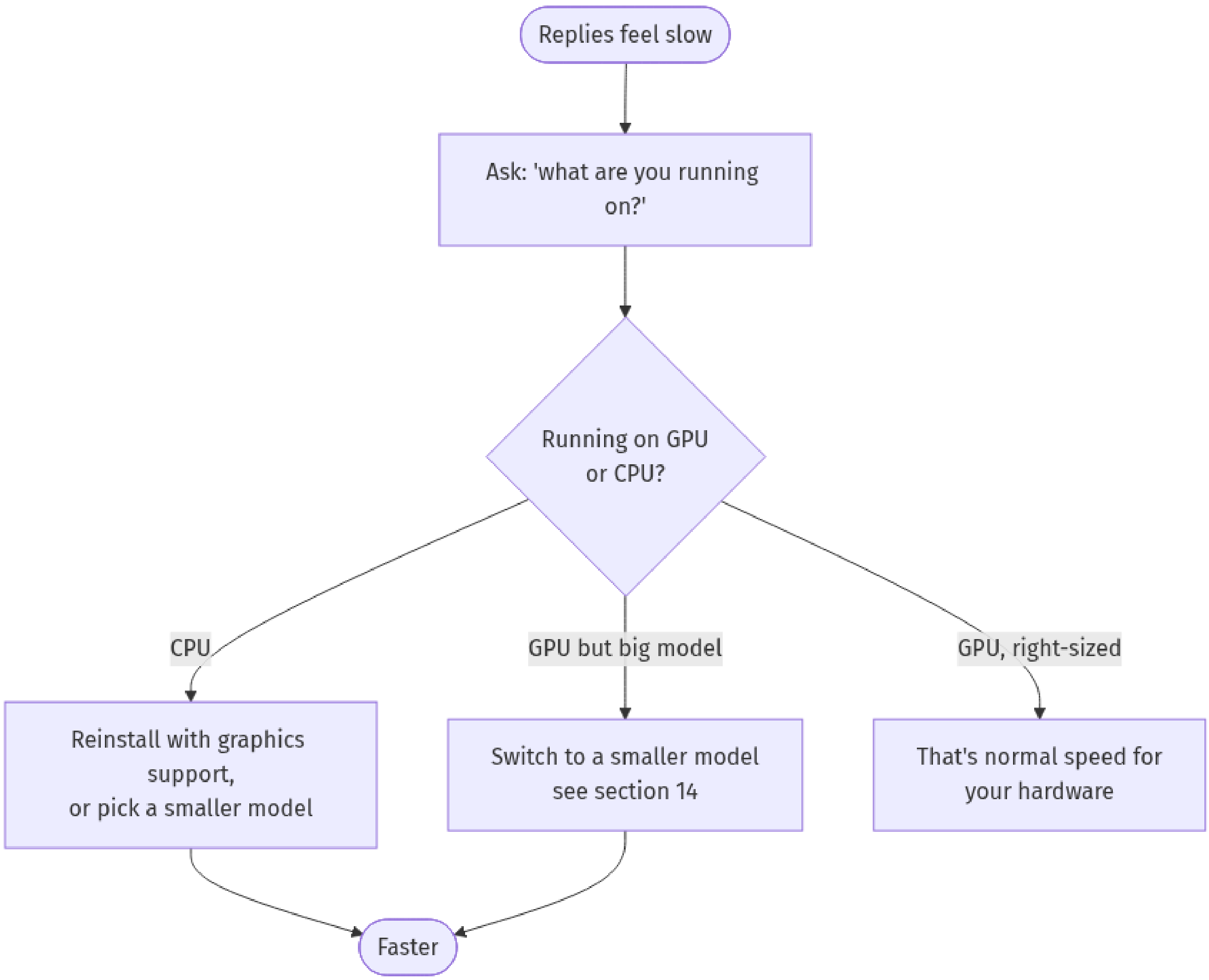
- **Nothing leaves your computer** unless you ask for something online (web search, news, weather, downloading a model). ELI tells you when it goes online.
- **Your data is yours** — conversations, memories, files, profile live in local databases. Delete anytime.
- **No accounts, no telemetry, no subscription.**
- A fresh install is a **blank slate** — ELI starts knowing nothing about you.

---

18. Troubleshooting

[A bit technical] Most issues map to one of these. When in doubt, **ask ELI** — “run a full runtime audit” or “status” — it reports honestly.

“ELI is replying very slowly”



Everything else

Problem	Try this
Replies look like gibberish	A model whose context is too small. Pick a recommended one (§14); ELI sizes context automatically.
“I couldn’t find a model”	`python -m eli.core.model_download --auto`
Memory/recall seems empty	Normal on a fresh install. Ensure the embedder downloaded: `python -m eli.core.model_download --aux`
Voice/wake word not responding	“run voice diagnostics”; check the microphone in Settings → Voice.
Won’t go online for news/search	By design (offline-first). The network is allowed per-request; check Settings → Privacy.
A habit didn’t run	Open **Habits** — confirm it’s enabled and the time is right; or “show my habits”.
Something seems broken	“run a full runtime audit” or “status”.

19. Quick phrasebook (cheat sheet)

Everyday - “open spotify and play lo-fi” · “pause” · “skip” - “what’s on my screen?” · “summarise report.pdf” - “set an alarm for 7am” · “set a timer for 10 minutes” - “what’s the news?” · “weather in Dublin” · “what time is it?”

Memory & you - “remember that …” · “what do you know about me?” · “my name is Alex”

Habits & proactive - “show my habits” · “yes” / “no” (to an offer) · “morning report” · “start proactive mode”

Writing & code - “write a report on X” · “write a script to …” · “fix the bugs in foo.py”

Voice - wake word “computer” + request · “change the wake word to athena” · “start dictation” · “say good morning” · “train my voice”

Check on ELI - “what are you running on?” · “status” · “what have you been working on?”

Models (in a terminal) - `python -m eli.core.model_download --list` · `python -m eli.core.model_download --auto`

---

20. Glossary

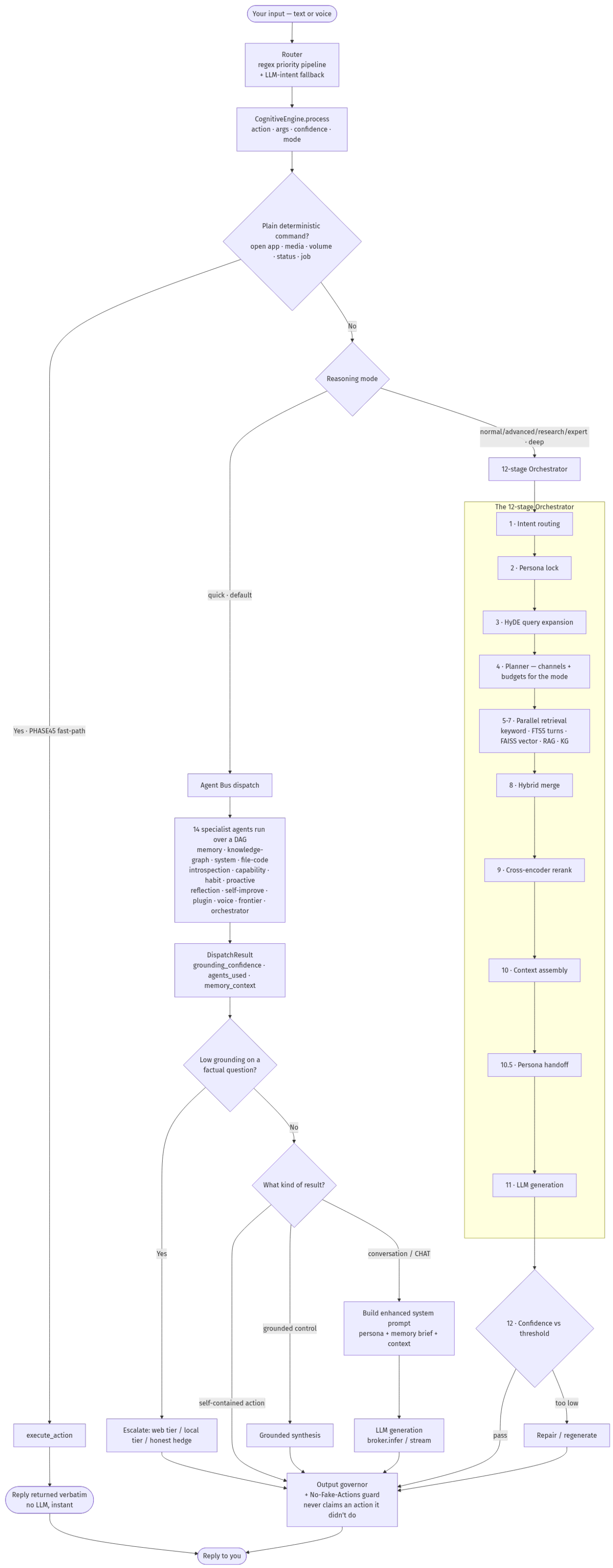
Term	Plain meaning
Model / GGUF	The “brain” file ELI thinks with. `.gguf` is just its file format.
VRAM	Memory on your graphics card. More VRAM → you can run a bigger, smarter model.
Embedder	A tiny helper model that powers ELI’s memory/recall. Installed automatically.
Context	How much of the conversation ELI can “hold in mind” at once. ELI sets this for you.
Vision model	The model that lets ELI understand screens and images.
Wake word	The phrase that makes ELI start listening (default “computer”).
Proactive mode	ELI doing helpful things on its own (briefings, suggestions).
Habit	A routine ELI learned and you approved, run on a schedule.
Fine-tuning	Optional advanced step to teach a model ELI’s voice (§14).
Offline-by-default	ELI never uses the internet unless you ask for something that needs it.

---

21. Appendix A — Under the hood: how ELI thinks (the full pipeline)

[Advanced] *For the curious / technical. You never need this to use ELI — but here is the complete, verified path from your words to ELI’s reply.*

Every request runs through one funnel — `CognitiveEngine.process()` — which takes one of three paths depending on what you asked and the reasoning mode. The diagram is the **real** pipeline (verified against the running code): a deterministic fast-path for plain commands, a **quick** path via the Agent Bus (the default), and a **deep** 12-stage Orchestrator for hard work.



Your input — text or voice
Router
regex priority pipeline
+ LLM-intent fallback
CognitiveEngine.process
action · args · confidence · mode
Plain deterministic command?
open app · media · volume · status · job
Yes · PHASE45 fast-path
No
Reasoning mode
quick · default
normal/advanced/research/expert · deep
12-stage Orchestrator
The 12-stage Orchestrator
1 · Intent routing
2 · Persona lock
3 · HyDE query expansion
4 · Planner — channels + budgets for the mode
5-7 · Parallel retrieval
keyword · FTS5 turns · FAISS vector · RAG · KG
8 · Hybrid merge
9 · Cross-encoder rerank
10 · Context assembly
10.5 · Persona handoff
11 · LLM generation
12 · Confidence vs threshold
pass
too low
Repair / regenerate
Agent Bus dispatch
14 specialist agents run over a DAG
memory · knowledge-graph · system · file-code introspection · capability · habit · proactive reflection · self-improve · plugin · voice · frontier · orchestrator
DispatchResult
grounding_confidence · agents_used · memory_context
Low grounding on a factual question?
Yes
No
What kind of result?
self-contained action
grounded control
conversation / CHAT
Build enhanced system prompt
persona + memory brief + context
Escalate: web tier / local tier / honest hedge
Grounded synthesis
LLM generation
broker.infer / stream
execute_action
Reply returned verbatim
no LLM, instant
Output governor
+ No-Fake-Actions guard
never claims an action it didn't do
Reply to you

The retrieval stages (5–7) are where memory plugs in — that's Appendix B. The two guarantees baked into the end of every path: the **Output governor** cleans the reply, and the **No-Fake-Actions guard** ensures ELI never claims it did something it didn't.

A.1 — The router (before any path is chosen)

Your words first hit the **router**. It runs a **regex-first priority pipeline**: an ordered set of fast, exact checks (web realtime lookups, media controls, file/PDF, memory, OS control, …), each carefully shaped so it only fires on a real command — e.g. media controls need a whole word in command shape, so "metabolise" never triggers a media "tab". If nothing matches, an **LLM intent classifier** is the fallback. The router emits `{action, args, confidence}` and hands off to `CognitiveEngine.process()`. The router also remembers the *last used path* so follow-ups ("do it again", "the second one") resolve correctly.

A.2 — The three paths, and when each is taken

- **Fast-path (PHASE45) — no AI involved.** Plain, deterministic actions (open an app, pause music, set volume, report status, check a job) are executed directly and the result is returned **verbatim**. There's no model call, so these are instant and can't be "hallucinated". This is why "pause" never produces a chatty paragraph.
- **Quick path (the default) — the Agent Bus.** Conversational and lightly-grounded requests go to the **Agent Bus**, which runs up to **14 specialist agents** in parallel over a dependency graph, each contributing evidence (memory hits, knowledge-graph facts, system readings, code search, capability info, …). Their combined result carries a **grounding confidence** — an honest "is this actually backed by anything?" score.
- **Deep path — the 12-stage Orchestrator.** When you ask for depth (advanced/research/expert modes, or a hard question), ELI runs the full pipeline below: more retrieval, reranking, and a confidence check with automatic repair.

A.3 — The Agent Bus, in full

The bus **selects a relevant subset** of agents for your intent (or fans out broadly when unsure), runs them concurrently, and **aggregates** their findings into a `DispatchResult`. Key agents and what they bring:

Agent	Contributes
Memory	recalls facts/turns from SQLite + full-text + vector search
Knowledge-Graph	entities and how they relate (who/what/where)
System	runs executor actions (web search, audits, status)
File-Code	searches the actual codebase for grounded answers about ELI itself
Introspection	live runtime/self state
Capability	what ELI can do (the capability manifest)
Habit / Proactive / Reflection / Self-Improve	routines, signals, insights, fixes
Plugin / Voice / Frontier / Orchestrator	plugin dispatch, speech state, deeper reasoning hooks, planning

After the bus returns, a **grounding-escalation hook** checks: *is this a factual question that came back weakly supported?* If so, ELI escalates — a web tier (if you allow online), a local tier, or an **honest hedge** ("I'm not certain, but…") rather than a confident guess. Then the result is finalised: a self-contained action returns directly, a grounded control action gets a short grounded synthesis, and a conversation goes to generation with the full persona + memory context.

A.4 — The 12 stages, one by one (the deep path)

1. **Intent routing** — settle exactly what you're asking for.
2. **Persona lock** — fix ELI's identity/voice for this turn so it stays consistent.
3. **HyDE query expansion** — ELI drafts a *hypothetical answer* and searches with **that** too, which dramatically improves what memory retrieves (you find better matches by searching with an answer-shaped query, not just the bare question). Skipped for very short queries.
4. **Planner** — decide *which* retrieval channels to use and how big the budgets are, tuned to the reasoning mode (fast / balanced / deep). 5–7. **Parallel retrieval** — several searches run at once: keyword, **full-text (FTS5)** over your conversation history, **FAISS vector** (semantic) search, optional RAG, and the **knowledge graph**. (This is the seam where Appendix B's memory system feeds in.)
5. **Hybrid merge** — combine the hits from every channel into one candidate set.
6. **Cross-encoder rerank** — a smarter model re-scores the merged candidates for true relevance to your question, and the best rise to the top. (Skipped for non-conversational asks.)
7. **Context assembly** — build the grounded context block that will inform the answer. 10.5. **Persona handoff** — wrap that context with ELI's live persona brief for generation.
8. **LLM generation** — the model writes the reply (streaming or one-shot). For the deepest modes this is a two-part private-reasoning-then-condense step.
9. **Confidence check** — score the answer against a threshold; if it's too low, ELI **repairs/regenerates** rather than shipping a weak answer.

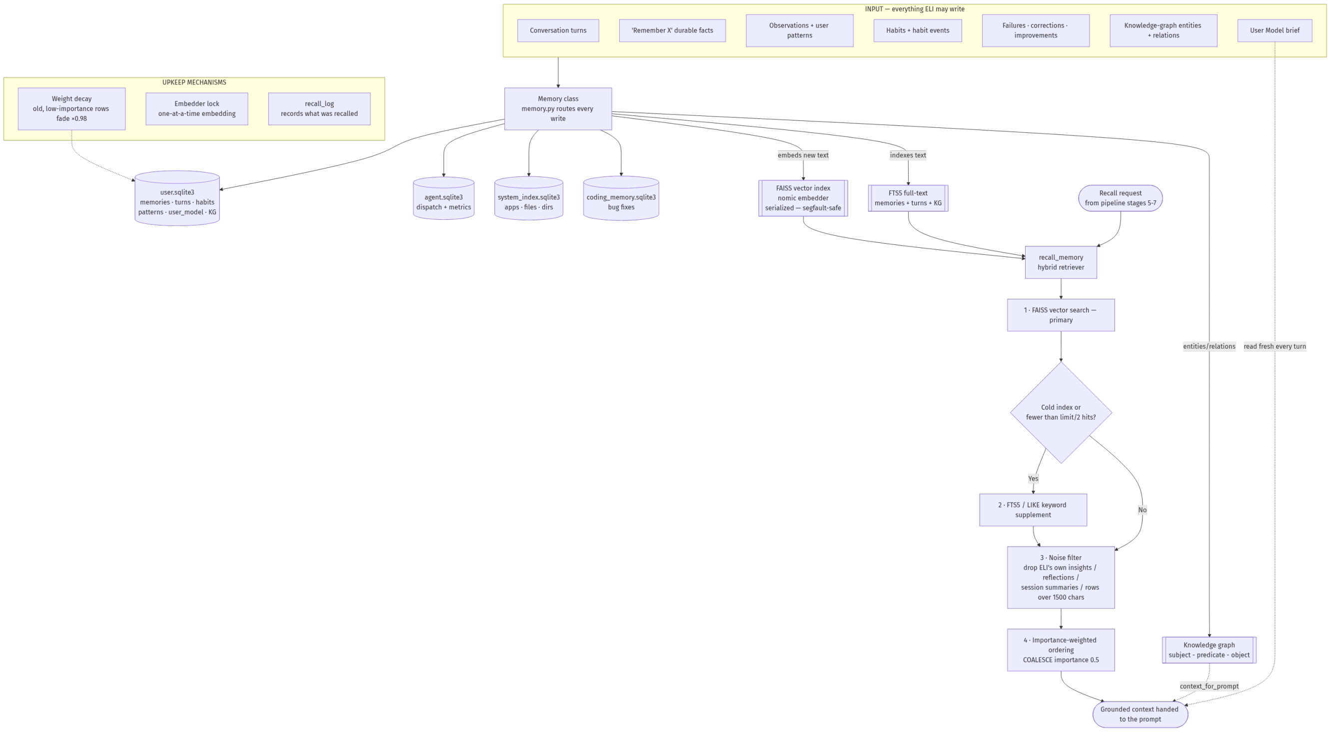
A.5 — The two end-of-path guarantees

Every path, no matter which, ends at: - **Output governor** — normalises and cleans the reply (strips artefacts, enforces formatting). - **No-Fake-Actions guard** — if ELI's reply *claims* it did something (played a song, fixed a file), the system verifies it actually happened; if not, the claim is replaced with the honest truth. This is enforced in code, not left to the model's goodwill.

---

22. Appendix B — Under the hood: how ELI remembers (the full memory system)

[Advanced] *For the curious / technical. ELI's memory is a genuine hybrid — relational + full-text + vector + graph — all local. This is the complete input → storage → recall path (verified against the code).*



Why it's trustworthy: the **noise filter** stops ELI's own past replies and reflections from resurfacing as if they were *your* facts; the **embedder lock** keeps the vector engine from crashing under concurrent use; and **weight decay** is the gentle "forgetting" that lets stale topics fade while active ones stay fresh. Nothing here leaves your computer.

B.1 — What ELI writes (the inputs)

Everything that flows into memory passes through one **Memory** module, which decides where each item belongs: - **Conversation turns** — what you said and what ELI replied, with timestamps. - **Durable facts** — anything you explicitly ask it to "remember". - **Observations & user patterns** — quiet signals about your interests, focus, and tone that build the living profile. - **Habits & habit events** — your repeated actions and the routines learned from them. - **Failures, corrections, improvements** — ELI's own mistakes and fixes, kept for self-learning (these are deliberately **excluded** from normal recall — see the noise filter). - **Knowledge-graph entities & relations** — named things and how they connect. - **User Model brief** — a compact, pre-rendered summary of who you are, refreshed over time.

B.2 — Where it's stored (four local databases + three indexes)

Everything is plain local files on your machine — no server, no cloud: - **`user.sqlite3`** — the main store: memories, conversation turns, habits, patterns, the User Model, and the knowledge graph. - **`agent.sqlite3`** — ELI's internal agent activity and performance metrics. - **`system_index.sqlite3`** — an index of your apps, executables, files, and folders (so "open the file manager" is instant). - **`coding_memory.sqlite3`** — remembered code bug-fixes.

On top of the relational data sit three search indexes: - **FAISS vector index** — every stored text is turned into an "embedding" (a list of numbers capturing meaning) by a small local **nomic embedder**, so ELI can find things by *meaning*, not just exact words. Embedding is done **one-at-a-time** behind a lock (the embedder isn't thread-safe; serialising it prevents crashes). - **FTS5 full-text index** — fast exact/keyword search over memories, conversation turns, and the knowledge graph. - **Knowledge graph** — a subject → predicate → object web (e.g. *you* → *own* → *a Tesla*) that ELI can walk to answer relational questions.

B.3 — How recall works (step by step)

When the cognition pipeline (stages 5–7) needs memory, it calls **`recall_memory`**, a genuine hybrid retriever that runs in this order: 1. **Vector search first (FAISS).** The semantic index is the primary path — it finds things that *mean* the same even if worded differently. 2. **Keyword supplement (FTS5/LIKE) — only if needed.** If the vector index is cold/empty or returns fewer than half the requested results (e.g. a very short query), a keyword search tops it up. (When another part of the system already did the vector search, this step is skipped to avoid double-searching.) 3. **Noise filter.** Results are scrubbed of ELI's *own* output — assistant insights, reflections, session summaries, the "orchestrator" source, and any over-long blobs (>1500 chars). This is the safeguard that stops ELI's past musings from masquerading as *your* facts. 4. **Importance-weighted ordering.** What survives is ranked by an importance score so the most significant memories surface first.

Alongside this, the **knowledge graph** contributes a lightweight relational context, and your **User Model brief** is read **fresh every single turn** — that's why ELI stays in context about who you are without you repeating yourself.

B.4 — Upkeep (the mechanisms that keep memory healthy)

- **Weight decay** — old, low-importance memories are gently faded (multiplied down over time), so abandoned topics naturally recede while active ones stay prominent. This is ELI's "forgetting".
- **Embedder serialization** — the single lock around the embedder that keeps the vector engine stable under concurrent access.
- **Recall log** — a record of what was retrieved, used for tuning and for honestly explaining *where* a piece of knowledge came from ("how do you know that?").

B.5 — Privacy, restated

Every database and index in this appendix is a local file under your user profile. There is no sync, no upload, no account. Delete the files (or use the **Memory** tab) and that knowledge is gone. A fresh install starts with the full structure but **zero** personal rows — a true blank slate.

That's the whole assistant — inside and out. You don't need any of the last two appendices to use ELI; just talk to it in plain English, and ask "what can you do?" whenever you're curious. Everything here happens on your computer, for you, privately.